École supérieure en Sciences et Technologies de l'Informatique et du Numérique

Module: Cryptographie Avancée
Niveau: 2CS – Cyber-Sécurité

Chapitre 4
Preuve à divulgation nulle de connaissance
&
Calcul multiparité sécurisé

Présenter par : DR BOUCHOUCHA

Année Universitaire 2024-2025

Introduction

Qu’est-ce qu’une preuve à connaissance nulle ?

Une preuve à connaissance nulle (ou **zero-knowledge proofs**, souvent abrégée en “ZKP”) est un protocole cryptographique qui peut **améliorer la confidentialité des données**.

Il permet à une partie, appelée le **fournisseur de preuve (prouveur),** d’assurer à une autre partie, **le vérificateur**, qu’une déclaration est vraie sans révéler l’information elle même.

Dans toute preuve à connaissance nulle, deux parties sont impliquées:

- Le **prouveur :** Il s'agit de l'entité qui souhaite prouver la validité d'une certaine affirmation sans révéler l'information elle même.

- **Le vérificateur** : Il s'agit de l'entité qui a besoin de s'assurer de la validité et l'affirmation, mais qui n'a pas nécessairement besoin de connaître les détails .

L'interaction entre ces deux rôles constitue le cœur d'une preuve à connaissance nulle.
Cela se faire par le biais d'une série d'interactions ou d'une seule chaîne d'informations, selon le type de preuve à connaissance nulle utilisé.

Les objectifs d’une preuve à connaissance nulle sont:

- Protection de la vie privée.
- Elle garantit qu’aucune information supplémentaire n’est exposée, ce qui en fait un outil précieux dans les protocoles cryptographiques.
- **Sécuriser les transactions et les vérifications de données sans compromettre la confidentialité.**

- Validation de l’authenticité d’une affirmation sans révéler aucune des données. Cela a des implications importantes pour diverses applications, **de la protection de l’identité** à la **conduite de transactions financières confidentielles**.

Un protocole à divulgation nulle doit satisfaire aux critères suivants :

1. **Complétude :** Si l'entrée est valide, le protocole à divulgation nulle renvoie toujours « vrai ».

2. **Validité :** Si l'entrée est invalide, il est théoriquement impossible de tromper le protocole à divulgation nulle pour qu'il renvoie la valeur 'vrai’.

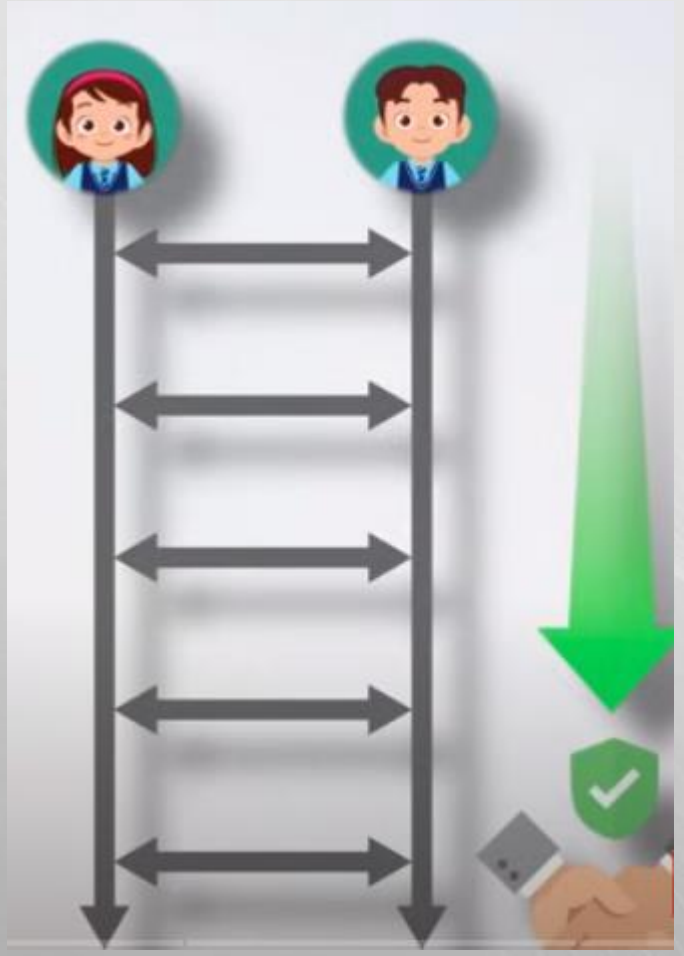
3. **Divulgation nulle :** Le vérificateur n'apprend rien de plus concernant une déclaration que sa validité ou sa fausseté. Cette condition empêche également le vérificateur de déduire l'information originale (le contenu de la déclaration) à partir de la preuve.

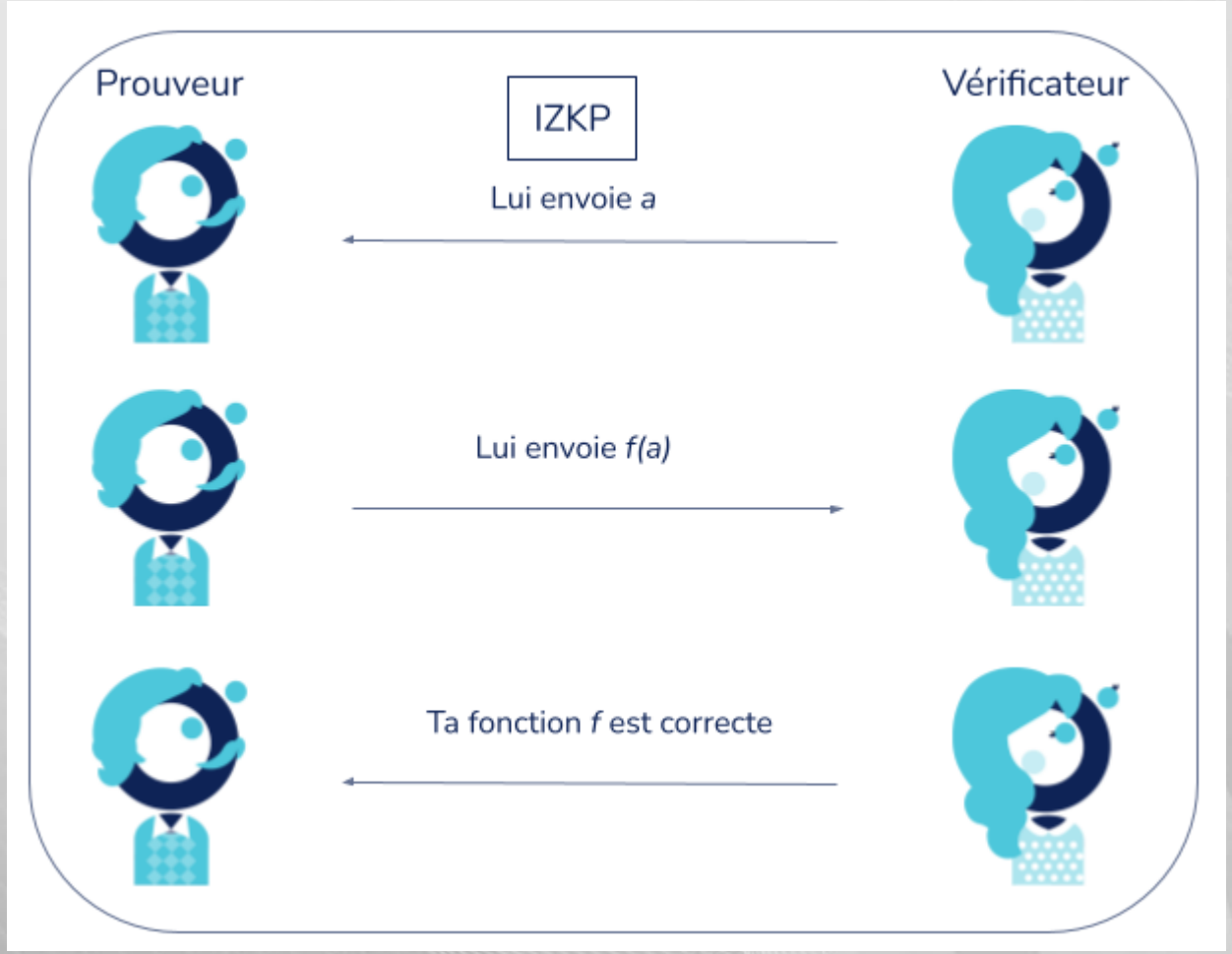
De manière générale, une preuve à divulgation nulle est **composée de trois éléments** :

•**Le témoin :** Dans le cas d'une preuve à divulgation nulle de connaissance, le prouveur veut prouver qu'il connaît certaines informations cachées. L'information secrète est le "témoin" de la preuve, et la connaissance supposée du témoin par le prouveur établit un ensemble de questions auxquelles seule une partie connaissant les informations peut répondre. Le prouveur commence donc le processus de preuve en choisissant aléatoirement une question, en calculant la réponse, et en l'envoyant au vérificateur.

•**Interrogatoire :** Le vérificateur choisit aléatoirement une autre question dans l'ensemble et demande au prouveur d'y répondre.

•**Réponse :** Le prouveur accepte la question, calcule la réponse et la renvoie au vérificateur. La réponse du prouveur permet au vérificateur de vérifier si le prouveur a réellement accès au témoin. Pour s'assurer que le prouveur ne devine pas à l'aveuglette et n'obtient pas les bonnes réponses par hasard, le vérificateur choisit davantage de questions à poser. En répétant cette interaction de nombreuses fois, la possibilité pour le prouveur de simuler la connaissance du témoin diminue significativement jusqu'à ce que le vérificateur soit satisfait.




Prouveur
IZKP
Vérificateur
Lui envoie a
Lui envoie f(a)
Ta fonction f est correcte

2. Types de preuves à connaissance nulle

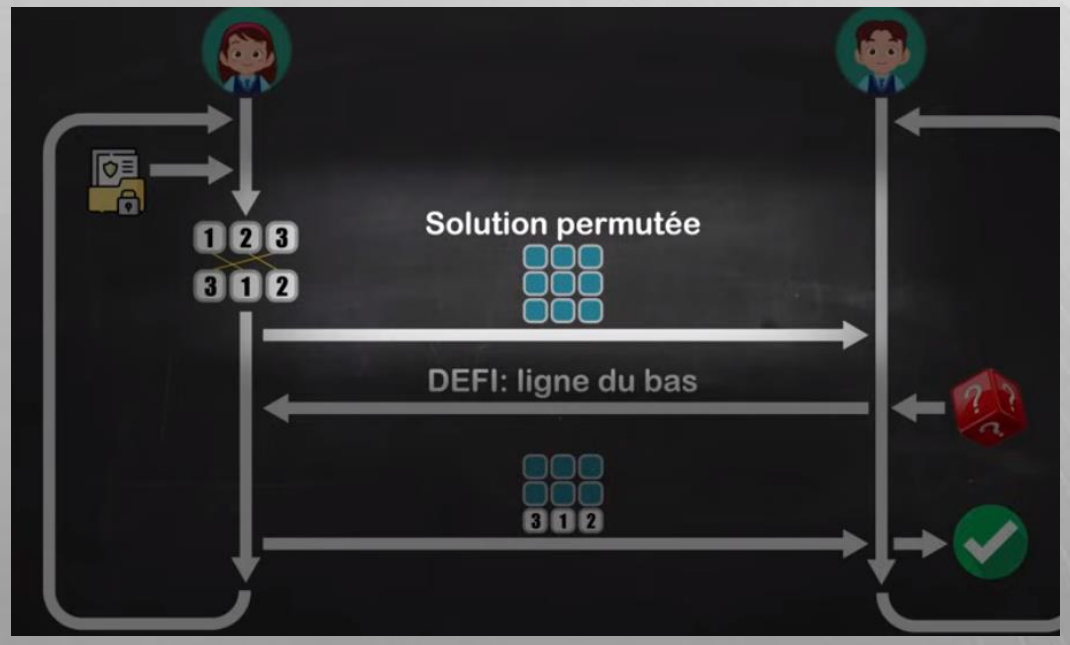
Les preuves à connaissance nulle ne constituent pas une solution unique.

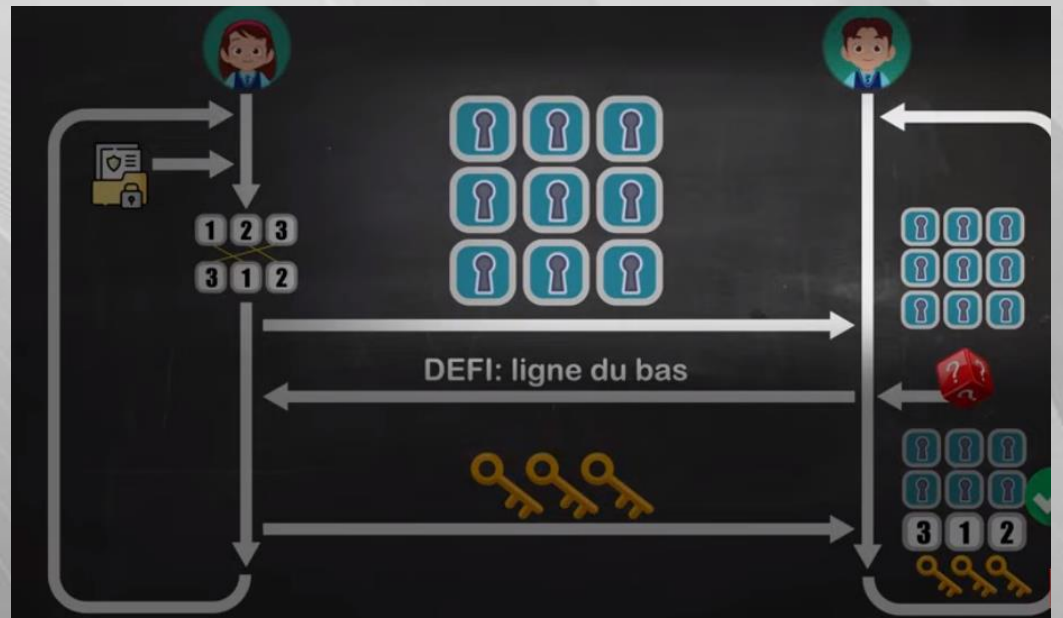
Il en existe plusieurs variétés, chacune ayant ses propres caractéristiques et applications.

2.1. Preuves interactives à zéro connaissance

Dans les preuves interactives à zéro connaissance, le prouveur et le vérificateur s'engagent dans un **dialogue en va-et-vient**. Cette interaction est essentielle pour que le vérificateur soit convaincu de la validité de l'affirmation. Bien qu'efficace, la nature interactive peut constituer une limite dans certains scénarios.

Avantages : Haut niveau de sécurité, bien adapté aux applications en temps réel
Inconvénients : nécessite plusieurs tours d'interaction, n'est pas idéal pour les systèmes asynchrones.





2.2. Preuves à connaissance nulle non interactives

Comme leur nom l'indique, les preuves non interactives sans connaissance **éliminent la nécessité d'un dialogue entre le prouveur et le vérificateur**. Un seul message du prouveur suffit pour convaincre le vérificateur.

Avantages : Efficace et évolutif, parfait pour la blockchain et d'autres systèmes décentralisés.

Inconvénients : légèrement moins sûr que les preuves interactives à zéro connaissance.

2.3. Zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) : Les zk-SNARKs combinent le meilleur des deux mondes. Ils permettent de prouver qu'une affirmation est **vraie en offrant une seule preuve très courte et rapide à vérifier (**des courbes elliptiques et des paires bilinéaires**)**. Ils sont non interactifs, mais assurent un haut niveau de sécurité, ce qui les rend populaires pour diverses applications, y compris la technologie de la chaîne de blocs.

Avantages : Hautement sécurisé, efficace et ne nécessitant aucune interaction
Inconvénients : configuration complexe et exigences de calcul plus élevées.

2.4. Zk-STARKs : Preuve transparente et sécurisée post quantique

Les zk-STARKs offrent un niveau de transparence que les zk-SNARKs n’offrent pas. Ils ne nécessitent pas de configuration de confiance (reposent sur les fonctions de Hachage), ce qui les rend plus transparents et potentiellement plus sûrs contre les attaques quantiques.

Avantages : Aucune configuration de confiance n’est requise, résistance aux attaques quantiques, grande évolutivité.
Inconvénients : taille des preuves plus importante, frais généraux de calcul plus élevés.

3. Cas d'utilisation des preuves à divulgation nulle de connaissance

1. Gestion sécurisée des identités

Les preuves de zéro connaissance peuvent révolutionner la façon dont nous gérons les identités en ligne. Imaginez que vous vous connectiez à un site de commerce électronique et que vous fassiez un achat sans que le site ne connaisse jamais vos véritables informations d’identification. Vous êtes vérifié, mais vos données personnelles restent avec vous.

2. Confidentialité financière

Dans le secteur bancaire, pensez à une demande de prêt immobilier pour laquelle la banque confirme que vous remplissez tous les critères financiers sans que vous ne révéliez vos revenus. Les preuves de connaissance zéro pourraient faire de ce niveau de confidentialité financière une réalité.

La Sécurité des données de santé Avec les ZKP, votre dispositif portable pourrait envoyer des mesures de santé vitales à votre prestataire de soins sans exposer votre identité ou d’autres données sensibles. C’est la transmission de données de santé avec une couche supplémentaire de confidentialité.

3. Confidentialité de l’IA et de l’apprentissage automatique

Les entreprises pourraient utiliser des preuves à connaissance nulle pour former des modèles d’intelligence artificielle sur des données chiffrées et multi-sources. Par exemple, la recherche pharmaceutique pourrait être accélérée par la mise en commun sécurisée de données provenant de plusieurs prestataires de soins de santé

4. Inconvénients à utiliser

1. Coûts en matériels

La génération de preuves à divulgation nulle de connaissance implique des calculs très complexes qu'il est préférable d'effectuer sur des machines spécialisées. Comme ces machines coûtent cher, elles sont souvent hors de portée des personnes ordinaires. En outre, les applications qui veulent utiliser une technologie à divulgation nulle doivent prendre en compte les coûts matériels, et cela peut augmenter les coûts pour les utilisateurs finaux.

2. Coûts de la vérification de la preuve

La vérification des preuves nécessite également un calcul complexe et augmente les coûts de mise en œuvre de la technologie à divulgation nulle dans les applications. Ce coût est particulièrement pertinent dans le contexte de la preuve du calcul.

3. Menaces liées à l'informatique quantique

ZK-SNARK utilise la cryptographie sur les courbes elliptiques (ECDSA) pour le chiffrement. Bien que l'algorithme ECDSA soit actuellement sûr, le développement d'ordinateurs quantiques pourrait briser son modèle de sécurité à l'avenir. Le ZK-STARK est considéré comme immunisé contre la menace de l'informatique quantique, car il utilise des hachages résistants à la collision pour le chiffrement. Contrairement aux appairages de clés public-privé utilisés dans la cryptographie sur les courbes elliptiques, le hachage résistant aux collisions est plus difficile à briser pour les algorithmes de calcul quantique.

5. Protocole de Fiat et Shamir

L'algorithme de Fiat-Shamir est une méthode permettant de mettre en œuvre un protocole de preuve à divulgation nulle de connaissance (ZKP).

Description du protocole

Le protocole s'effectue entre un prouveur A et un vérifieur B. **Etant donné un entier naturel n**, le prouveur veut montrer qu'il connaît une **racine carré S d'un entier I** (donnée initiale) sans pour autant la divulguer. Le protocole s'effectue **itérativement L fois**.

- Le prouveur choisit un entier naturel inversible de manière aléatoire suivant une distribution uniforme dans **Bn** et calcule son carré x modulo n.
- Le prouveur envoie **x** au vérifieur.
- Le vérifieur choisit un bit **e** de manière aléatoire suivant une distribution uniforme.
- Le vérifieur envoie **e** au prouveur.
- Le prouveur calcule $\boldsymbol{Y = r\, S^e \bmod n}$ et l'envoie au vérifieur.
- Le vérifieur vérifie l'égalité $Y^2 = x\, I^e \bmod n$.

Paramètre :	n un entier
Donnée secrète :	$S \in \mathbb{Z}_n{}^*$
Donnée initiale :	$I = S^2 \bmod n$

<u>Répéter ℓ fois</u> :

Prouveur		Vérifieur
$r \in_R \mathbb{Z}_n{}^*$		
$x = r^2 \bmod n$	$\xrightarrow{\quad x \quad}$	
	$\xleftarrow{\quad e \quad}$	$e \in_R \{0, 1\}$
$y = r \times S^e \bmod n$	$\xrightarrow{\quad y \quad}$	vérifier $y^2 \stackrel{?}{=} x \times I^e \bmod n$

Le protocole a recours à plusieurs reprises à des choix probabilistes, et le résultat lui-même est probabiliste : le vérifieur ne pourra jamais être totalement sûr de la connaissance d'une racine carré de I par le prouveur.

- si e = 0, le prouveur n'a pas besoin de connaître le secret pour donner une réponse valide. On pourrait en déduire que le vérifieur devrait toujours choisir **e = 1**, mais alors il suffirait au prouveur de choisir $\boldsymbol{x = \frac{Y^2}{I}}$ pour obtenir une réponse valide sans pour autant connaître le secret.
- Une étape suivante dans le raisonnement serait alors de demander les deux résultats $\boldsymbol{Y_0 = r \bmod n}$ et $\boldsymbol{Y_1 \equiv r \times S \bmod n}$, mais alors il suffirait au prouveur de calculer $\boldsymbol{Y_0^{-1}\, Y_1 \bmod n}$ **pour obtenir S** et la preuve cesserait d'être à divulgation nulle de connaissance.
- S'il est impossible d'obtenir une certitude sur la connaissance de la racine d'un nombre, le vérifieur peut affiner sa certitude autant qu'il le veut en **augmentant le nombre d'itérations. En effet, la probabilité que le prouveur connaisse le secret est de** $\boldsymbol{\frac{1}{2^L}}$**.**

6. De la preuve au système d'authentification

Une fois un protocole interactif de preuve à divulgation nulle de connaissance trouvé, le passage à un système d'authentification est immédiat.

Reprenons donc le protocole de Fiat et Shamir. **Il fut que chaque utilisateur choisisse un nombre I modulo un grand nombre n, et fournisse le couple ($I^2 \ mod \ n, n$) lors de son inscription sur un site.**

Lors du processus d'authentification, il suffirait que l'utilisateur entre son secret dans son navigateur (supposé honnête), lequel interagirait avec le site distant suivant le protocole de Fiat et Shamir jusqu'à ce que ce dernier soit convaincu de la connaissance par l'utilisateur du secret, et donc de son identité.

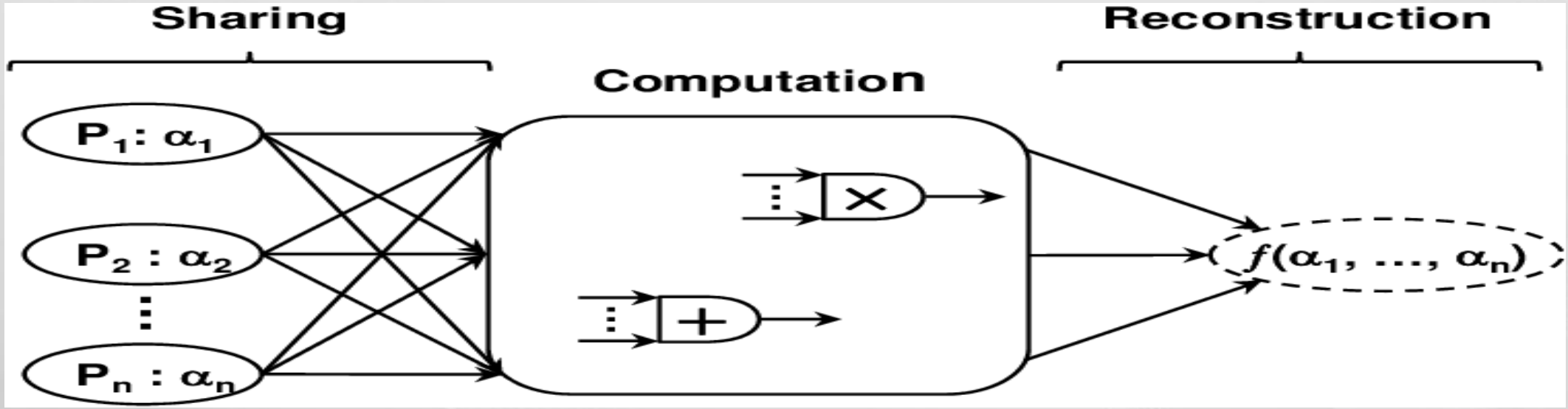
7. Le calcul multipartite sécurisé (*Secure Multi-Party Computation*))

Le calcul multipartite sécurisée est un protocole cryptographique qui évalue un calcul entre plusieurs parties– chacune détenant ses propres données privées –où aucune partie individuelle ne peut voir les données des autres parties.

Les deux propriétés de base qu'un protocole de calcul multipartite doit garantir sont :

•**Confidentialité :** Les informations privées détenues par les parties ne peuvent être déduites de l'exécution du protocole.

•**Précision/ Exactitude :** si un certain nombre de parties au sein du groupe décident de partager des informations ou de s'écarter des instructions pendant l'exécution du protocole, le MPC ne leur permettra pas de forcer les parties honnêtes à produire un résultat incorrect ou à divulguer les informations secrètes d'une partie honnête.



Il existe plusieurs approches techniques, mais voici quelques grands principes :

•**Secret sharing** (partage de secret) : chaque donnée est divisée en morceaux (parts) et distribuée entre les participants. Aucune part seule ne permet de reconstituer la donnée.

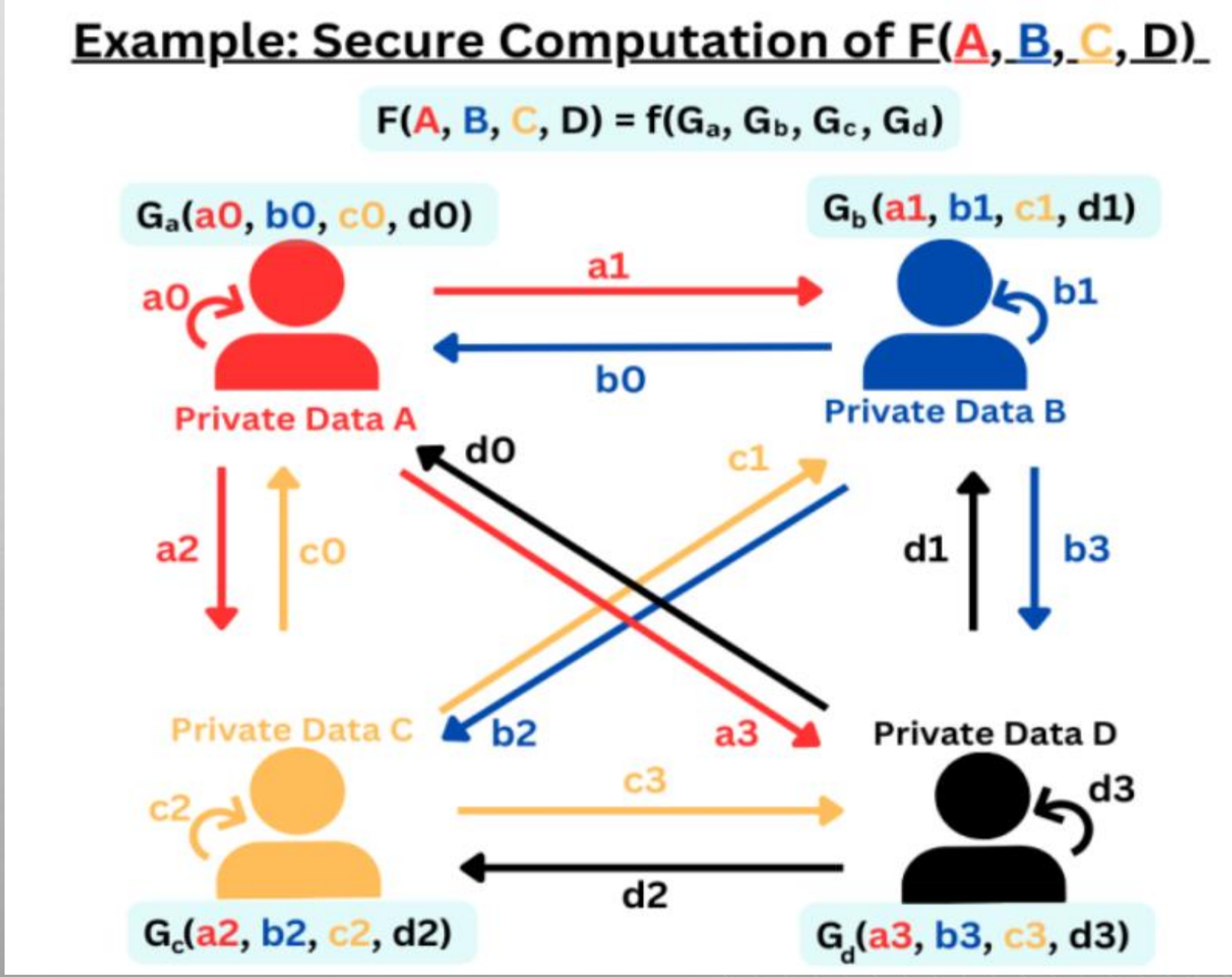
•**Homomorphic encryption** : les calculs sont faits directement sur des données chiffrées.

•**Garbled circuits** : un protocole cryptographique où une des parties encode un circuit logique pour qu'une autre puisse l'exécuter sans rien apprendre.

Le but de MPC est de fournir un protocole dans lequel Jane, John , Bobby et Jack peuvent apprendre F(x, y, z,u) sans révéler qui fait quoi et sans avoir besoin de s'appuyer sur un autre tiers en échangeant des messages exclusivement entre eux.

Ils ne devraient pas acquérir plus de connaissances en participant à leur procédure qu'en
converser avec un incorruptible, totalement digne de confiance.

F(x,y,z,u)=max(x,y,z,u)



La fonction F peut être calculée à l'aide de divers protocoles SMPC (Secure Multi-Party Computation), comme:

1. les circuits tronqués de Yao
2. le partage de secrets de Shamir
3. le protocole ABY2.0 .

De plus, chaque protocole SMPC est conçu pour répondre aux conditions essentielles de confidentialité et d’exactitude.

8. Avantages du calcul multipartite sécurisé

Les avantages du calcul multi partite sécurisé sont:

1.Tiers de confiance : dans Secure Multiparty Computation, nous pouvons partager des données de manière distribuée avec différentes organisations sans aucun tiers et même la confidentialité des données sera préservée lors du partage de données.

2.Confidentialité des données : les données privées des organisations peuvent être partagées à des fins de calcul. Le souci de confidentialité des données est assuré par l’utilisation d’un calcul multipartite sécurisé, qui conserve les données utilisées sous forme cryptée. Ainsi, les données ne sont ni révélées ni compromises.

3.Haute précision : le calcul multipartite sécurisé fournit des résultats très précis pour différents calculs utilisant la cryptographie.

4.Sécurité quantique : les données partagées entre les parties sont protégées contre les attaques quantiques, car les données sont divisées et cryptées lorsqu'elles sont distribuées entre les parties à des fins de calcul.

9. Limites du calcul multipartite sécurisé

Le calcul multipartite sécurisé est utilisé pour résoudre différents problèmes, mais il existe peu de limitations.

Les principales limites sont :

1.Surcharge de calcul : pour fournir la sécurité dont nous avons besoin pour générer les nombres aléatoires, ces derniers nécessite davantage de surcharge de calcul, ce qui ralentit l'exécution.

2. Coûts de communication élevés : la distribution des données à plusieurs parties pour le calcul sur les réseaux entraîne des coûts de communication plus élevés.

10. Domaine d’application:

- La vente aux enchères de contrats: Dans cette application, seule l'offre gagnante finale sera divulguée à tous les participants.

- Analyser les disparités salariales basées sur le sexe et l'origine ethnique au sein des organisations

- Comparaison et l'analyse des données ADN privées d'un individu avec une base de données d'ADN de patients atteints de cancer pour le diagnostic du cancer

11. Transfert inconscient

Le transfert inconscient est une primitive très importante en cryptographie moderne.

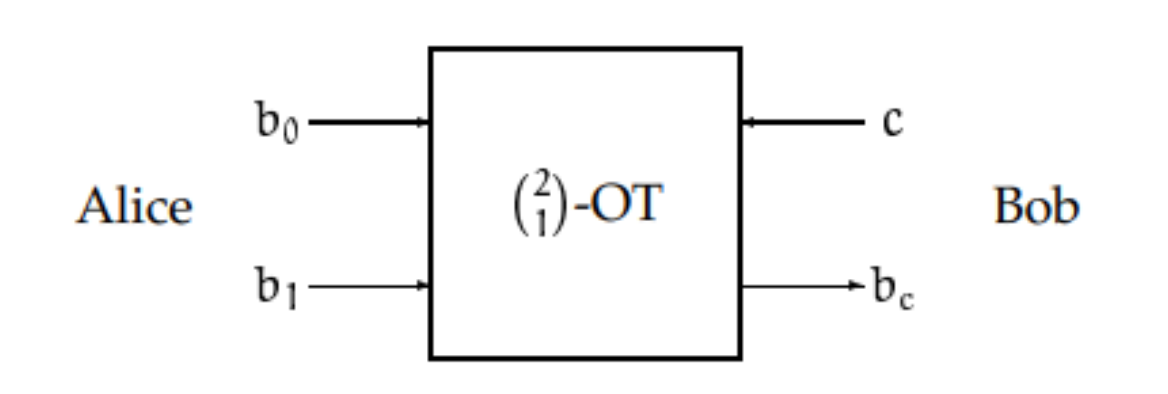
Il permet d’accomplir une foule de tâches cryptographiques:

- La mise en gage de bit ,
- Les preuves interactives,
- Les calculs multi-parties sur données privées.

Supposons qu'Alice connaisse (choisisse) deux bits b0 et b1 et que Bob choisisse un bit C et veuille apprendre le bit bc .

Nous ajoutons cependant quelques restrictions :

- Alice est prête à donner un de ses bits à Bob à la condition que Bob n'apprenne aucune information sur l'autre bit.
- Plus formellement, nous dirons que Bob n'apprend aucune information sur le bit bc , même si Alice lui donne, après le transfert, le bit bc.
- Bob est prêt à participer à la condition qu'Alice n'apprenne rien sur son choix, c'est-à-dire son bit bc.



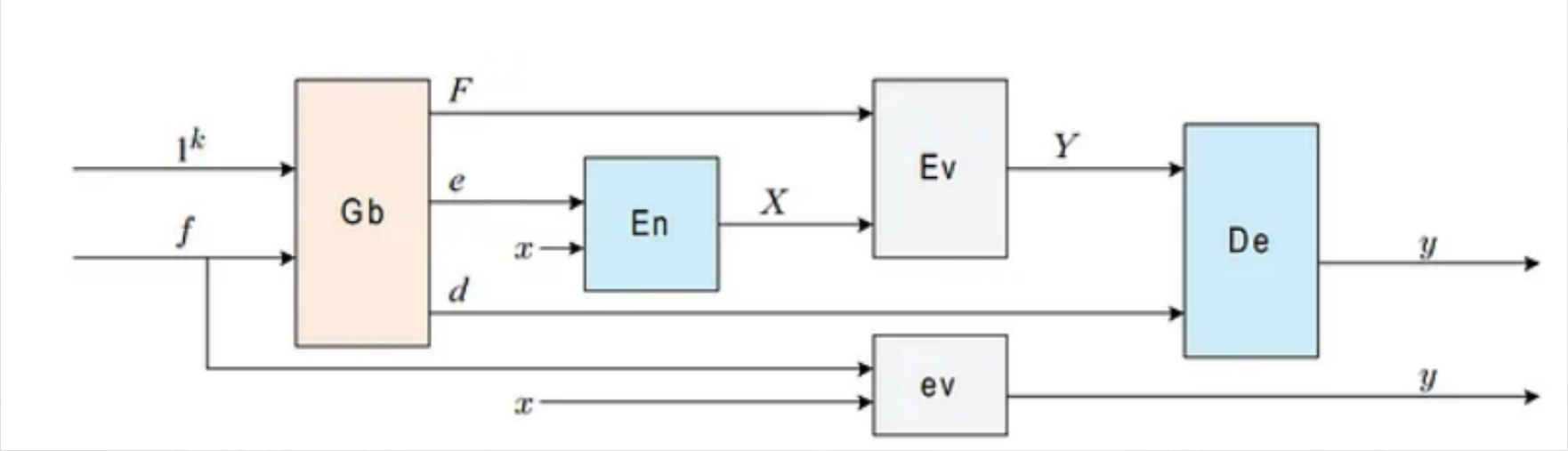
12. Protocole basé sur Les circuits tronqués de Yao

Le circuit tronqué est un protocole cryptographique qui permet un calcul bipartite sécurisé. Il permet à deux parties méfiantes d'évaluer conjointement une fonction sur leurs contributions privées sans l'implication d'un tiers de confiance. La fonction doit être définie comme un circuit booléen dans le protocole de circuit tronqué.

12.1. Protocole de circuit tronqué

La fonction sous-jacente, telle que la fonction de comparaison dans le problème des millionnaires, est décrite comme un circuit booléen avec des portes à 2 entrées. Les deux parties connaissent le circuit, qui peut être fourni au préalable par un tiers.

1. Alice brouille (crypte) le circuit, assumant le rôle de la brouilleuse.
2. Alice envoie le circuit tronqué à Bob avec son entrée cryptée.
3. Bob doit tronquer son entrée pour calculer le circuit, ce qui nécessite l'aide d'Alice puisque seul le brouillé sait chiffrer. Bob utilise un transfert inconscient pour chiffrer ses entrées. Alice est l'expéditeur et Bob est le destinataire de ce transfert.
4. Bob décrypte le circuit pour obtenir les sorties cryptées, agissant en tant qu'évaluateur.
5. Alice et Bob communiquent pour connaître le résultat.



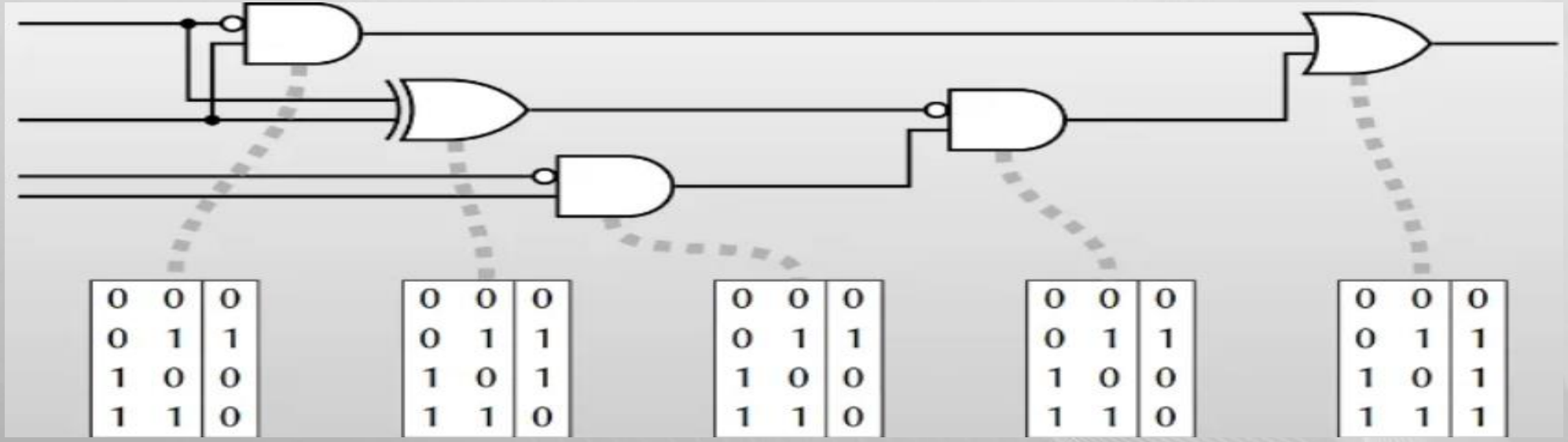
- **Un algorithme de garbling Gb** est un algorithme aléatoire qui transforme une fonction f : {0, 1}^n → {0, 1}^m en un triplet de fonctions (F, e, d) ← Gb(f) , tel que f = d ◦F ◦ e.
- **La fonction de codage e** convertit une entrée initiale x ∈ {0, 1}^n en une entrée tronquée X = e(x).
- L'évaluation de la fonction brouillée F sur l'entrée brouillée X produit une sortie brouillée Y = F(X).
- **La fonction de décodage d** transforme la sortie tronquée Y en la sortie finale y = d(Y), qui devrait idéalement correspondre à f(x).

Formellement, Gb n'est pas considéré comme agissant sur des fonctions, donc un schéma de garbling G est défini comme **un cinq-tuple d'algorithmes (Gb, En, De, Ev, ev**), où **les chaînes d, e, f et F sont interprétées comme fonctions sous les auspices des algorithmes De, En, ev et Ev.**

Le paramètre secret k est généralement fixé à 128.

12.2. Les circuits tronqués du Yao

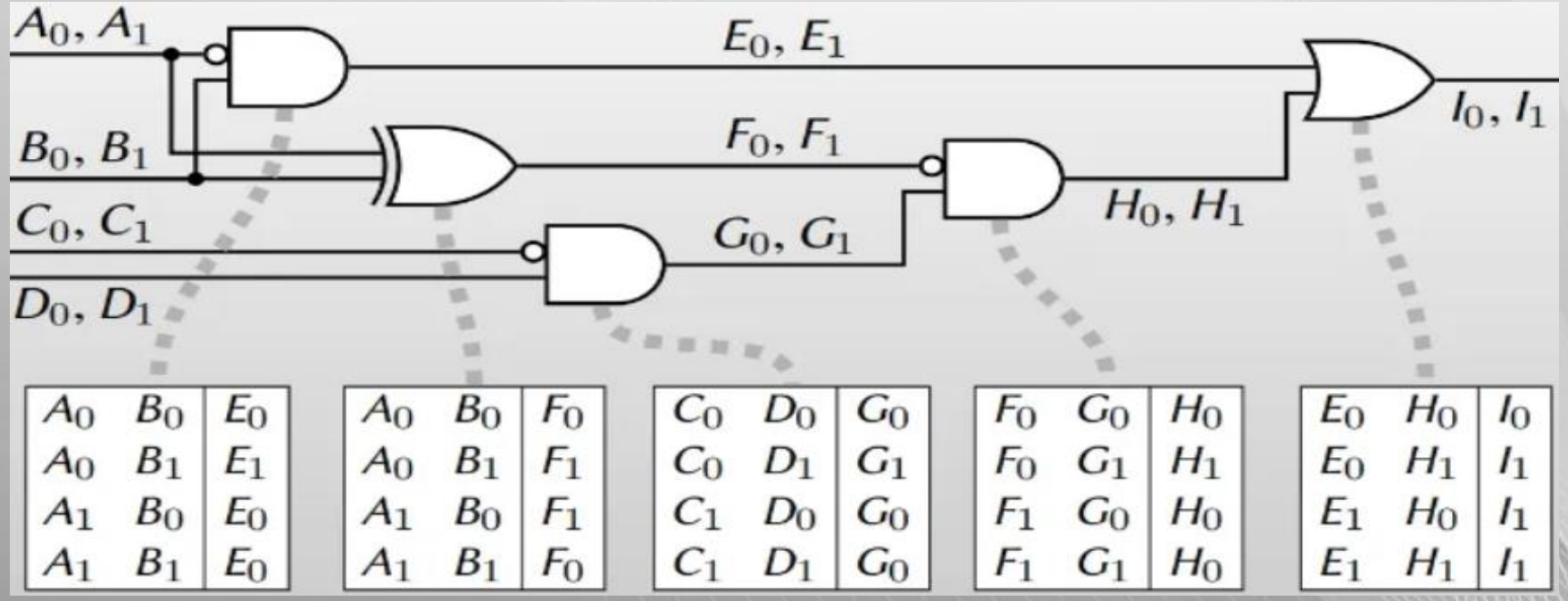
1.Alice crée la table de vérité basée sur le circuit.



2. Alice attribue **deux chaînes générées aléatoirement**, **appelées étiquettes**, à chaque fil du circuit : une pour la sémantique booléenne 0 et une pour 1.

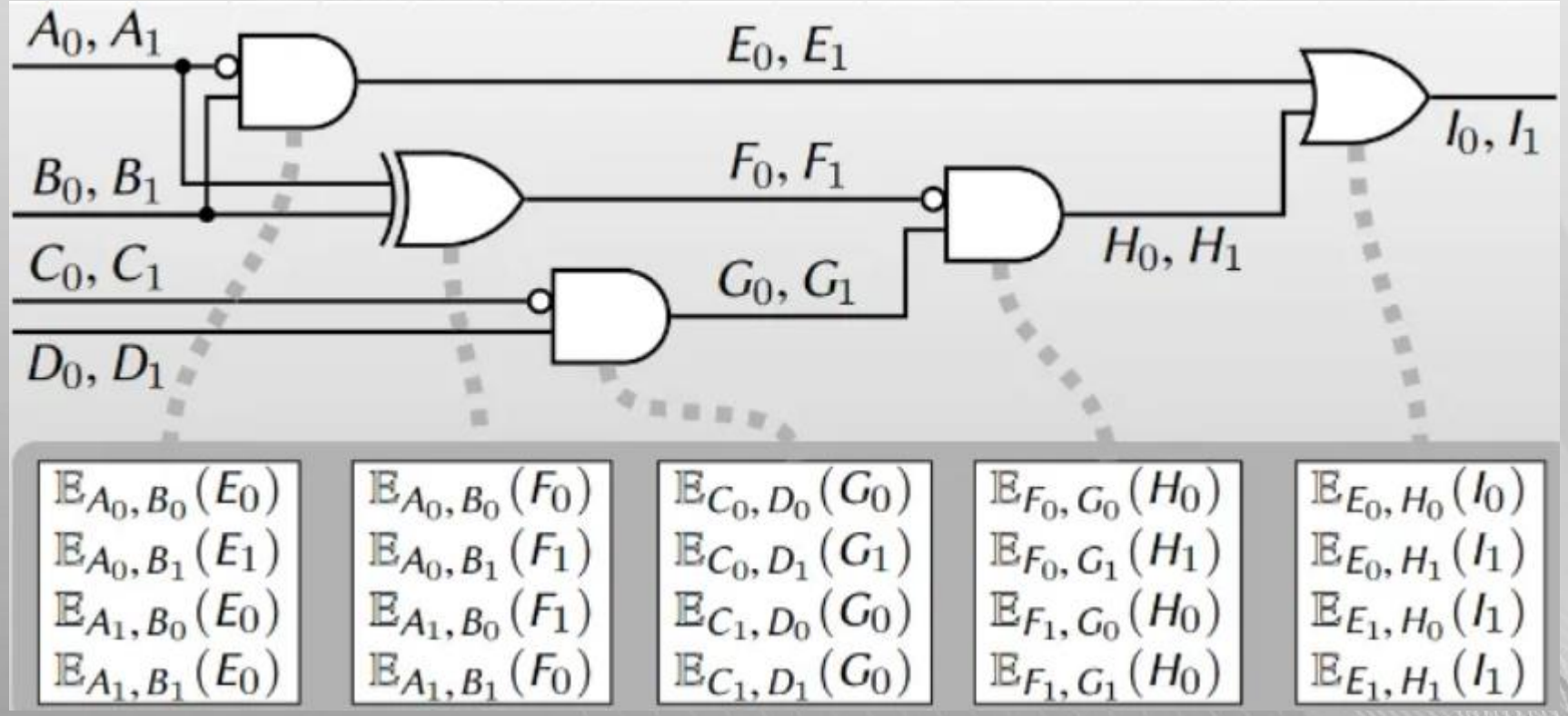
L'étiquette a généralement une longueur de k bits.

Alice passe ensuite à toutes les portes du circuit et remplace 0 et 1 dans les tables de vérité par les étiquettes correspondantes.

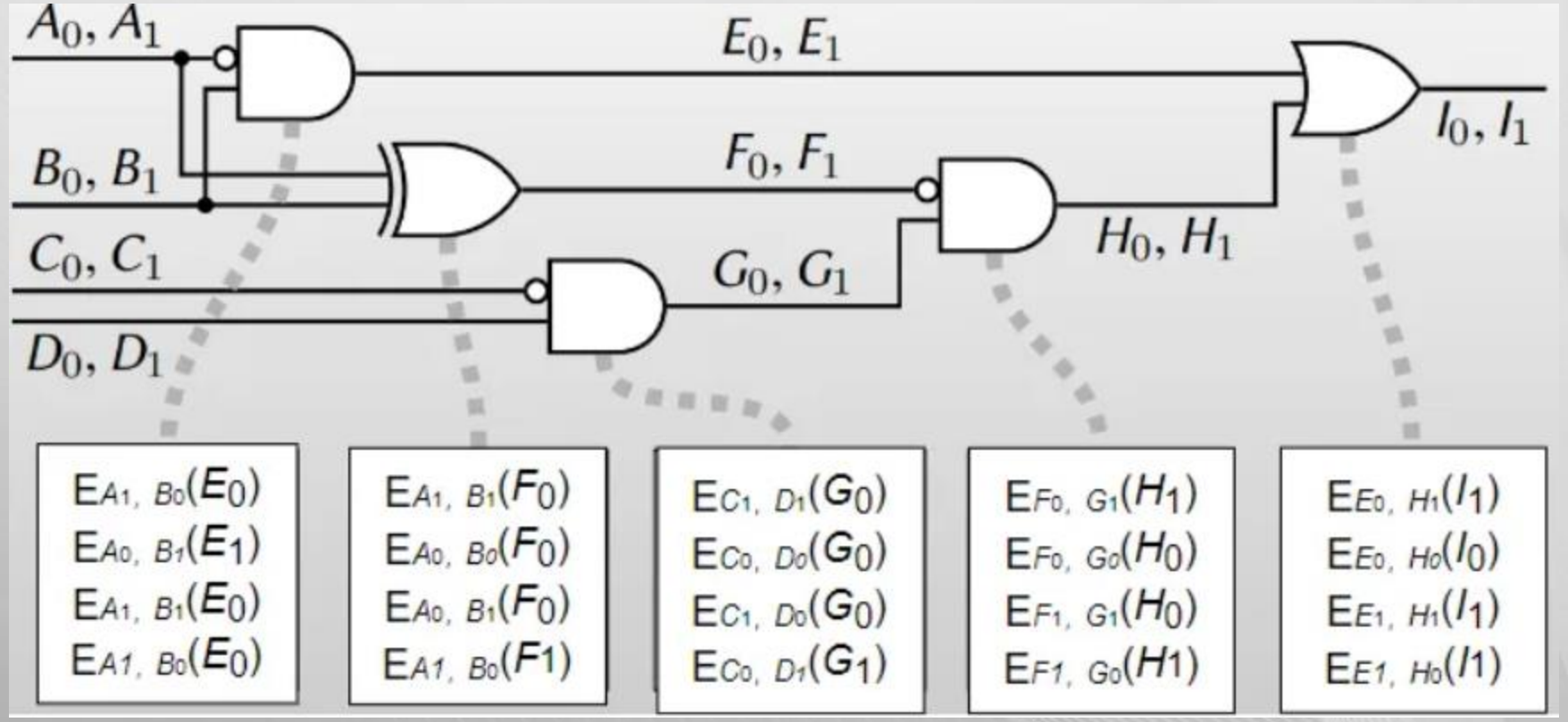


3. Alice chiffre ensuite l'entrée de sortie de la table de vérité en utilisant les deux étiquettes d'entrée correspondantes.

Cette table cryptée est la table tronquée, qui ne peut être déchiffrée que si l'on possède les deux étiquettes d'entrée correctes. Dans le tableau ci-dessous, un chiffrement symétrique à double clé est utilisé et **(A0, B0) représente la clé secrète**, tandis que **E0 désigne la valeur à chiffrer**.

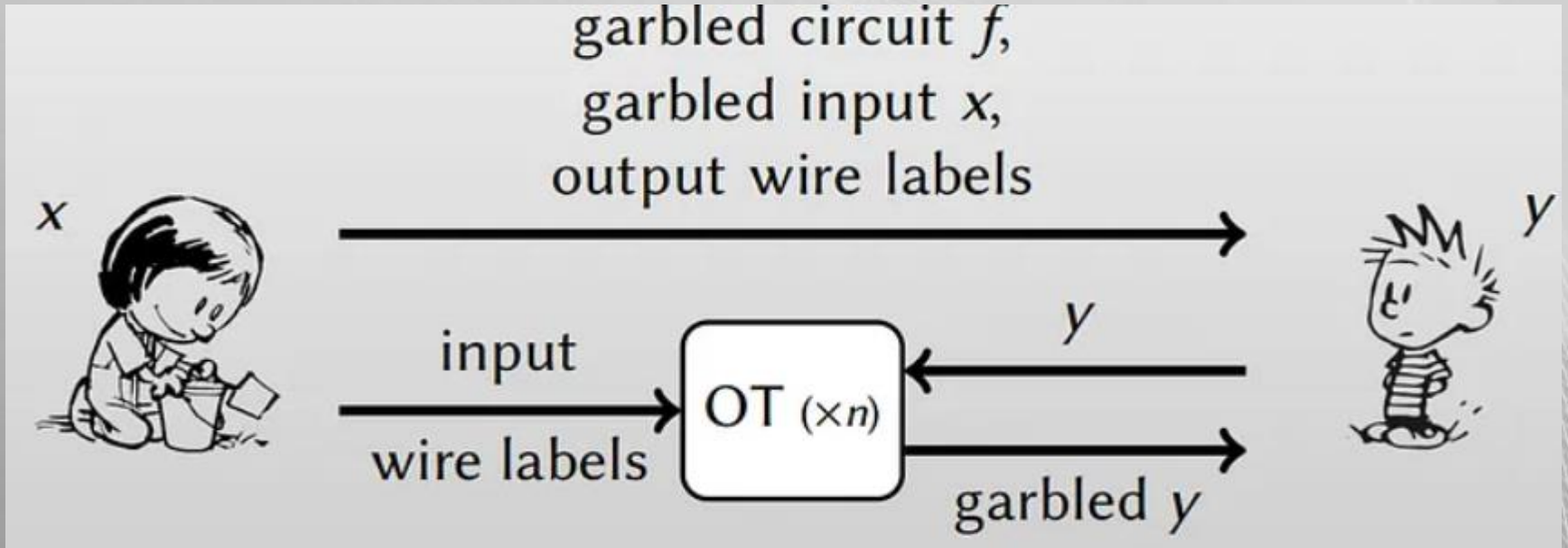


4. Alice permute ensuite le tableau de manière aléatoire de telle sorte que la valeur de sortie ne puisse pas être déterminée à partir de la ligne. Le nom du protocole, garbled , est dérivé de cette permutation aléatoire.

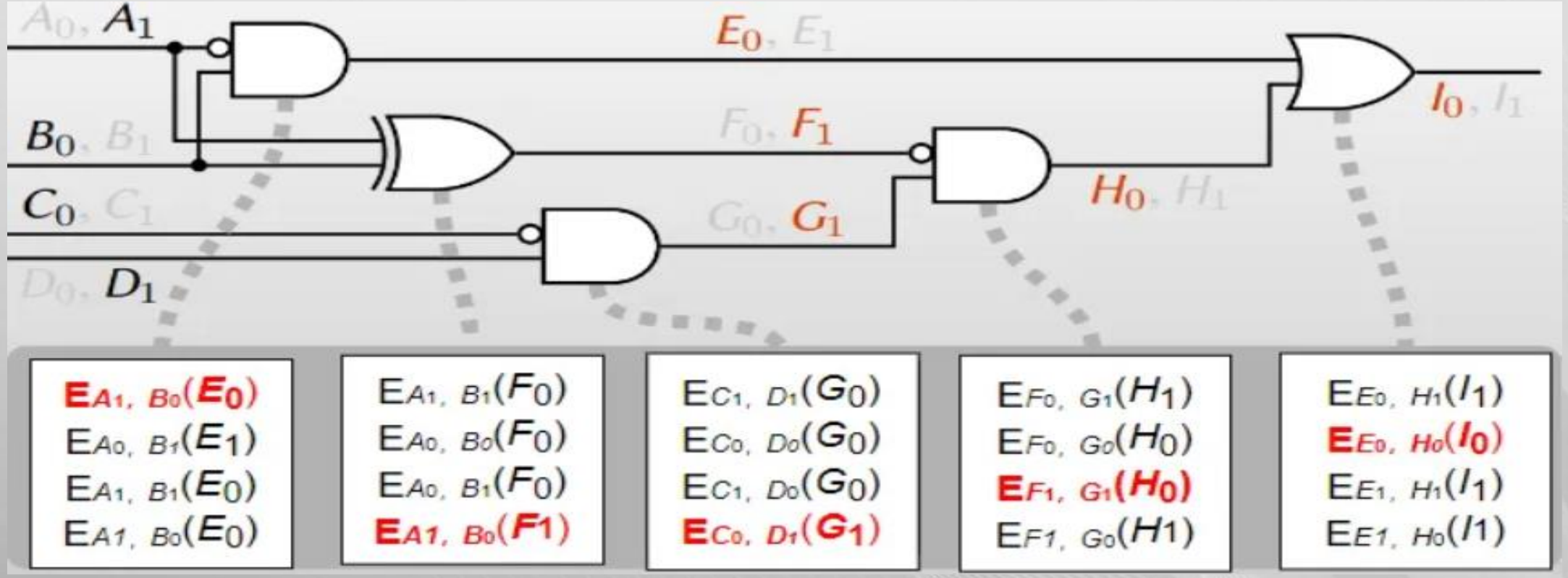


5. Alice envoie à Bob les tableaux tronqués calculés pour toutes les portes du circuit. Bob a besoin d'étiquettes d'entrée pour ouvrir les tables tronquées. Ainsi, Alice choisit les étiquettes correspondant à sa saisie et les envoie à Bob. ***Disons que les fils A et C sont les entrées d'Alice et que les fils B et D sont les entrées de Bob.***

Bob n'apprendra rien sur l'entrée d'Alice puisque Alice génère de manière aléatoire les étiquettes qui ressemblent à des chaînes aléatoires pour Bob. Bob a également besoin des étiquettes correspondant à son entrée. Il reçoit ses étiquettes via des transferts inconscients pour chaque bit de son entrée.
Après les transferts inconscients, Alice n'apprendra rien sur la contribution de Bob, et Bob n'apprendra rien sur les autres labels.



6. Après le transfert de données, Bob dispose des tableaux tronqués et des étiquettes d'entrée. Il franchit toutes les portes une à une et tente de décrypter les rangées de leurs tables tronquées. Il est capable d'ouvrir une ligne (ligne rouge) pour chaque table.



7. Après l'évaluation, Bob obtient l'étiquette de sortie (I 0), et Alice connaît son mappage à la valeur booléenne puisqu'elle a les deux étiquettes (I0, I1). Alice peut partager ses informations avec Bob, ou Bob peut révéler le résultat à Alice afin que l'un d'eux ou les deux apprennent le résultat.

13. Protocole de Partage secret de Shamir

Le partage secret d'Adi Shamir est un algorithme cryptographique qui permet à des parties distinctes de partager conjointement la propriété d'un seul secret en détenant des actions . Le secret d'origine ne peut être reconstitué qu'en utilisant un nombre minimum de partages, ce qui permet aux différentes parties de coopérer sans avoir besoin de se faire entièrement confiance.

13.1. Comprendre le seuil

Chaque système de partage Shamir a **un nombre total d'actions et un seuil**. Le seuil est le nombre de partages requis pour reconstituer le secret d'origine. Par exemple, avec cinq partages et un seuil de trois, vous n'avez besoin que de trois des cinq partages pour calculer le secret d'origine.

L'une des propriétés mathématiques fondamentales utilisées dans le partage secret de Shamir est le fait qu'il faut **k points pour définir un polynôme de degré k - 1.**

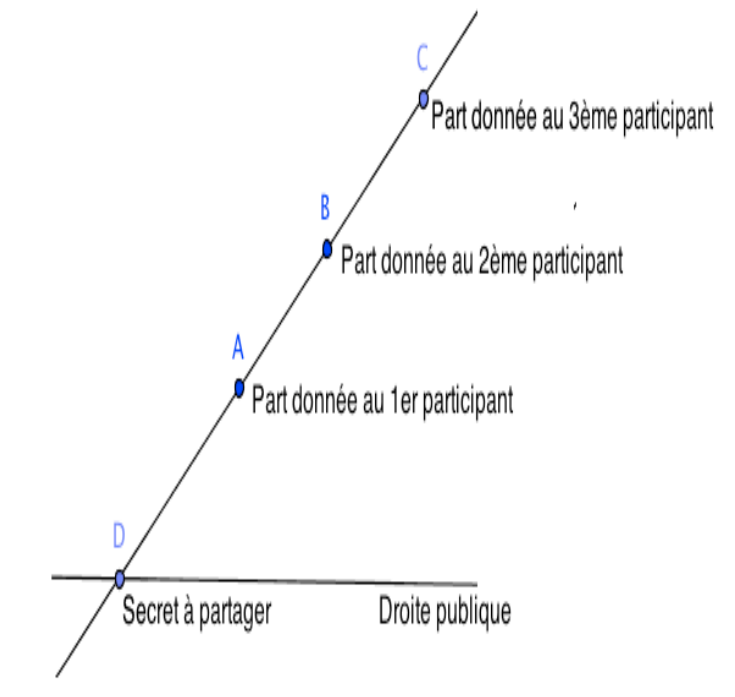
•Une seule ligne peut être tracée entre deux points

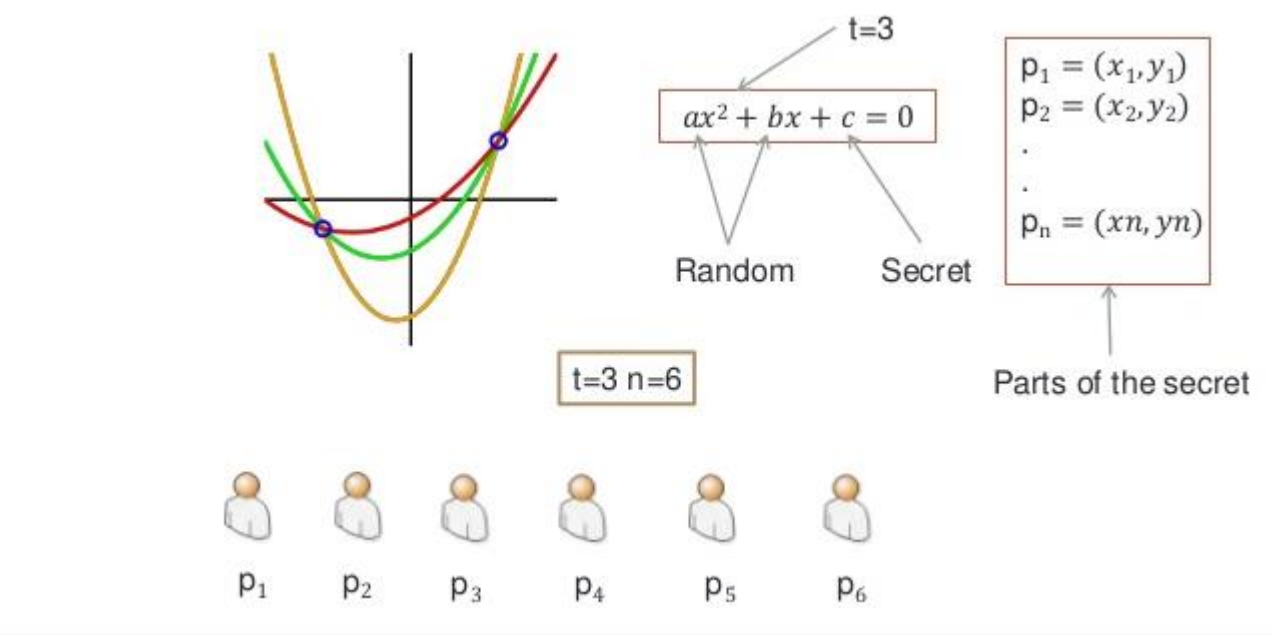
•Une seule parabole possible passe par les trois mêmes points

•Une seule courbe cubique passe par les quatre mêmes points

•Un nombre infini de lignes peut être tracé à travers le même point

•Un nombre infini de paraboles peut être tracé à travers les deux mêmes points





13.2. Système de partage de secret de Shamir

Afin de Construire un schéma de partager de secret (S) avec (n) partages et un seuil de (k) , il faut:

1. Choisir au hasard k - 1 entiers positifs (coefficients) dans un corps fini F;
2. Poser a0= S
3. Construire le polynôme: $f(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \ldots + a_{k-1} x^{k-1}$.

4. Ensuite, nous utilisons cette formule pour créer les points (les actions) que nous donnons à chaque participant.

13.3. Reconstitution du secret

on utilise deux méthodes :

1.Fixer le seuil et choisir les points définissent pour définir le types courbes et donner une valeur à x0.

2. Utiliser le polynôme de Lagrange:

$$f(x) = \sum_{j=0}^{2} y_j \ell_j(x) \ ,$$

où les ℓ_j sont les polynômes de base de Lagrange

END